🏛 MASTER 6-HOUR DAILY OPERATIONAL TRAINING & LAB BLUEPRINT

High Performance Computing — A Practical Approach (GLA-HPC-BCSG701)

Intel & NEC Supercomputing Alliance × Edulateral Foundation | GLA University, Mathura

Lead Trainer: Shivam Gupta | **Target Audience:** 7th-Semester B.Tech Computer Science & IT Students
Daily Schedule: 6 Full Hours / Day (3h Morning Architecture + 3h Afternoon Server/Colab Lab)
Strict Syllabus Authority: `Intel & NEC - High Performance Computing.pdf`

⏰ THE 6-HOUR DAILY PEDAGOGICAL FRAMEWORK

Every single day of the masterclass is divided into two 3-hour blocks designed to maximize retention, deep understanding, and hands-on mastery on university HPC servers and Google Colab:

```
+-------------------------------------------------------------------------------+
|                  THE 6-HOUR MASTER DAILY SCHEDULE (09:30 AM - 04:30 PM)       |
+-------------------------------------------------------------------------------+
| 🌅 MORNING BLOCK (09:30 AM - 12:30 PM | 3 Hours / 180 Minutes)                |
|    "Concept → System Understanding → Trainer Live Demo"                       |
|    • Slides from MAIN_PRESENTATIONS/ (15 slides per day, clean visuals)       |
|    • Whiteboard mathematical derivations (Amdahl's Law, Roofline model, Ring-AllReduce)  |
|    • Architectural deep-dive: Silicon registers, L1/L2/L3 caches, memory bus, fabrics    |
|    • Trainer live coding & syntax explanation on projector                    |
|    • Mid-session rejuvenation break (10 minutes at 11:15 AM)                  |
+-------------------------------------------------------------------------------+
| 🍱 LUNCH & RECOVERY BREAK (12:30 PM - 01:30 PM | 60 Minutes)                  |
+-------------------------------------------------------------------------------+
| 💻 AFTERNOON BLOCK (01:30 PM - 04:30 PM | 3 Hours / 180 Minutes)              |
|    "Hands-On Server Labs → Guided Coding Challenges → Verification & Assessment"  |
|    • Students login to University Linux Cluster Servers via SSH or Google Colab Sandbox |
|    • Step-by-step lab stages from 02_DAY_XX_HANDS_ON_LAB_AND_COMMANDS.md      |
|    • Live benchmarking: Measuring real hardware speedups on CPUs, GPUs, and clusters  |
|    • Troubleshooting common errors (permissions, segfaults, MPI slots, race conditions)  |
|    • 80-90% Instant Recall Cheat Sheet review & 25 Evaluated MCQs             |
+-------------------------------------------------------------------------------+
```

5-DAY MODULE-BY-MODULE ARROW WORKFLOW ROADMAP

```
================================================================================
DAY 1: Fundamentals of HPC, Linux & Shell Scripting
================================================================================
Module 1: Introduction to HPC (Why HPC → Serial vs Parallel → Amdahl's Law → Speedup)
Module 2: Intel HPC Hardware Architecture (Xeon CPU → Intel GPU → Habana Gaudi → Memory Wall)
Module 3: Linux Basics & VFS Navigation (Terminal → Filesystem Hierarchy → Permissions → Paths)
Module 4: Shell Scripting for Automation (Shebang → Variables → Redirection → hpc_node_audit.sh)
Module 5: Secured Shell (SSH) Remote Access (Keygen Ed25519 → authorized_keys → Host Verification)
Module 6: Intel oneAPI Toolkit Ecosystem (SYCL → DPC++ Compiler → Unified Shared Memory → icpx)
Module 7: Introduction to Intel HPC Toolkit (Compilers → OpenMP Primitives → MPI Cluster Hello)

================================================================================
DAY 2: Traditional ML using Intel oneAPI
================================================================================
Module 8: Parallel Computing Fundamentals (Task vs Data Parallelism → Shared vs Distributed Memory)
Module 9: OpenMP for Multithreading (#pragma omp parallel for → Reduction Clause → Thread Scaling)
Module 10: Hardware False Sharing & MESI Protocol (64-byte Cache Lines → alignas(64) → 4x Speedup)
Module 11: MPI for Distributed Systems (MPI_Init → Ranks & Communicators → Collective MPI_Reduce)
Module 12: Data Decomposition & Synchronization (Domain Chunking → Synchronization Overhead)
Module 13: Intel AI Analytics Toolkit & oneDAL (NumPy MKL → patch_sklearn() → 500k Sample K-Means 20x)

================================================================================
DAY 3: Deep Learning on HPC with TensorFlow, Slurm & GPUs
================================================================================
Module 14: ML Workflows on HPC Systems (Data Ingestion → Loss & Optimizer → Checkpointing)
Module 15: Hardware-Aware ML Optimization (Compute vs Memory Bound → Operator Fusion → Precision)
Module 16: Intel-Optimized TensorFlow & oneDNN (oneDNN Primitives → Conv+BN+ReLU Fusion in Cache)
Module 17: Supercomputer Accelerators: Intel GPU & Gaudi (MME Systolic Arrays → Integrated 100GbE)
Module 18: Slurm Workload Manager & Job Scheduling (slurmctld & slurmd → sbatch → #SBATCH Directives)
Module 19: Discrete-Event Slurm Simulation (Queue Prioritization → Partition Allocation → Logs)
Module 20: Distributed Training with Horovod (hvd.init() → DistributedOptimizer → Multi-Worker MPI)

================================================================================
DAY 4: Distributed AI Training & Profiling Optimization
================================================================================
Module 21: Distributed TensorFlow Training with Slurm (Multi-Node sbatch → RoCE / InfiniBand Fabrics)
Module 22: Ring-AllReduce Algorithmic Mechanics (Ring Step-by-Step Shift → 2*(N-1)/N * S Transfer Math)
Module 23: Intel VTune Profiler Hotspot Analysis (Unoptimized Matrix Kernel → Sampling Counters)
Module 24: Top-Down Microarchitecture Analysis (TMAM L1/L2: Retiring vs Frontend vs Backend vs Bad Spec)
Module 25: Intel Advisor Roofline Modeling (Arithmetic Intensity (FLOP/byte) → Memory vs Compute Ceiling)
Module 26: Cluster Scaling Laws (Amdahl's Fixed-Size Limit vs Gustafson's Scaled Speedup at 256 Cores)

================================================================================
DAY 5: Generative AI & LLMs on Intel HPC
================================================================================
Module 27: Introduction to Generative AI & LLMs on HPC (Transformer Math → KV-Cache → Memory Bandwidth)
Module 28: Intel Silicon for LLMs (Xeon AMX Matrix Tiles → Habana Gaudi 2/3 On-Chip 24x 100GbE Fabrics)
Module 29: Intel OpenVINO Toolkit IR Export (PyTorch/HuggingFace Model → .xml Graph + .bin Weights)
Module 30: NNCF INT4 Weight-Only Quantization (16-bit to 4-bit Compression → 72% RAM Footprint Reduction)
Module 31: Real-Time LLM Latency Streaming Benchmark (Time-to-First-Token (TTFT) → Tokens/Sec (TPS))
Module 32: Multi-Node Slurm Distributed LoRA Fine-Tuning (DeepSpeed ZeRO-2/3 → sbatch Cluster Script)
Module 33: High-Concurrency FastAPI Serving Microservice (Async REST API → /health & /api/generate)
```

📋 DAY 1: COMPLETE 6-HOUR OPERATIONAL PLAYBOOK

Theme: HPC Foundations, Intel Silicon, Linux Command Mastery, SSH Keys & oneAPI Discovery
Official Reference: Topics 1–7 on Page 2 of `Intel & NEC - High Performance Computing.pdf`

👷 Morning Session (09:30 AM - 12:30 PM | 3 Hours): Architecture & Concepts

- **09:30 - 10:15 (45m) | Module 1: Introduction to HPC & Scientific Parallelism**
  - *Arrow Tree:*

```
[ Massive Scientific/AI Workload ]
       |
       ▼
[ Hardware Bottleneck Audit ] ──► Single CPU: 60 Hours vs HPC Cluster: 15 Mins
       |
       ▼
[ Mathematical Problem Decomposition ] ──► Split Spatial Grid or Batch Rows
       |
       ▼
[ Parallel Execution & Speedup ] ──► Speedup S = T_serial / T_parallel
```

  - *Trainer Delivery:* Explain why laptops melt under heavy loads (Power Wall: $P = C \cdot V^2 \cdot f$, memory bandwidth choke). Contrast laptop 2-channel DDR4 (50 GB/s) with dual-socket Intel Xeon 16-channel DDR5 (600+ GB/s).
  - *Whiteboard:* Draw Amdahl's Law curve showing why a 5% serial portion limits maximum theoretical speedup to 20x.
- **10:15 - 11:00 (45m) | Module 2: Intel HPC Hardware Architecture Continuum**
  - *Arrow Tree:*

```
[ Workload Type ]
   ├── General Logic / Preprocessing ──► Intel Xeon Scalable CPU (Heavy cores, large caches, AMX)
   ├── Dense Vector / Image Tensors ──► Intel Data Center GPU Max (Massive SIMD threads)
   └── Large Transformer Training ──► Habana Gaudi 2/3 (Systolic MMEs + 24x 100GbE RoCE)
```

  - *Trainer Delivery:* Demystify the silicon. Walk through L1d (1ns), L2 (3.5ns), L3 (12ns), and DDR5 RAM (65ns). Explain why Gaudi integrates RDMA networking directly onto the accelerator package.
- **11:00 - 11:15 (15m) | ☕ Rejuvenation Break**
- **11:15 - 12:00 (45m) | Modules 3 & 4: Linux VFS Hierarchy & Shell Script Automation**
  - *Arrow Tree:*

```
[ User / Terminal ] ──► [ Shell / Bash ] ──► [ VFS Kernel (/proc, /sys) ] ──► [ Hardware Registers ]
```

  - *Trainer Delivery:* Walk through `/proc/cpuinfo`, `/proc/meminfo`, directory permissions (octal `755`, `700`), environment variables (`PATH`, `LD_LIBRARY_PATH`), and script execution mechanics (`#!/usr/bin/env bash`, `set -euo pipefail`).
- **12:00 - 12:30 (30m) | Modules 5, 6 & 7: SSH Remote Access, oneAPI & HPC Toolkit**
  - *Arrow Tree:*

```
[ Local Terminal ] ──(Ed25519 Handshake)──► [ Remote HPC Cluster Node ]
                                                        │
                                                        ▼
                                              [ source setvars.sh ]
                                                        │
                                                        ▼
                                              [ icpx / dpcpp Compilers ]
```

- *Trainer Delivery:* Explain public/private asymmetric cryptography. Introduce Intel oneAPI as an open unified cross-architecture programming model (SYCL) rather than just a compiler.

💻 Afternoon Session (01:30 PM - 04:30 PM | 3 Hours): Server & Colab Hands-On Lab

- **01:30 - 02:00 (30m) | Lab Stage 1 & 2: Cluster Hardware Audit & Linux Environment Setup**
  - Students SSH into university cluster servers (or open Colab notebook).
  - Execute `lscpu`, inspect NUMA topology (`numactl -H`), verify cache line sizes.
  - Create project directory hierarchy: `mkdir -p ~/hpc_day01/{scripts,logs,bin,data}` with strict permissions (`chmod 750`).
- **02:00 - 02:45 (45m) | Lab Stage 3 & 4: Production Bash Automation (`hpc_node_audit.sh`) & SSH Setup**
  - Students write and execute the automated cluster audit script (`hpc_node_audit.sh`):

```
cat << 'EOF' > ~/hpc_day01/scripts/hpc_node_audit.sh
#!/usr/bin/env bash
set -euo pipefail
echo "============================================================"
echo "         GLA UNIVERSITY HPC COMPUTE NODE AUDIT              "
echo "============================================================"
echo "Hostname       : $(hostname)"
echo "OS Kernel      : $(uname -r)"
echo "Active Cores   : $(nproc)"
echo "CPU Model      : $(lscpu | grep 'Model name:' | sed 's/Model name:[ \t]*//')"
echo "RAM Available  : $(free -h | awk '/Mem:/ {print $7}')"
echo "============================================================"
EOF
chmod +x ~/hpc_day01/scripts/hpc_node_audit.sh
~/hpc_day01/scripts/hpc_node_audit.sh
```

  - Generate SSH Ed25519 key pairs (`ssh-keygen -t ed25519 -N ""`) and configure passwordless access.
- **02:45 - 03:45 (60m) | Lab Stage 5, 6 & 7: Intel DPC++ / SYCL Vector Add & OpenMPI Multi-Process Ring**
  - Compile and execute the 10,000,000-element SYCL unified shared memory parallel kernel:

```
# Fail-safe dual-path compiler command
if command -v icpx >/dev/null 2>&1; then
    icpx -fsycl -O3 dpcpp_vector_add.cpp -o vector_add
else
    g++ -std=c++20 -O3 -fopenmp -march=native dpcpp_vector_add.cpp -o vector_add
fi
./vector_add
```

  - Compile and execute the 4-process OpenMPI cluster communication ring:

```
mpicxx -O3 mpi_cluster_hello.cpp -o mpi_cluster_hello
mpirun --allow-run-as-root --oversubscribe -n 4 ./mpi_cluster_hello
```

- **03:45 - 04:30 (45m) | Lab Stage 8: Cheat Sheet Review & 25 Evaluated MCQs**
  - Review the Day 1 Cheat Sheet for 80-90% instant recall.
  - Conduct the 25-question student assessment with live explanations.

📌 DAY 1 "ISKO LAGA DAALA TOH 80-90% KNOWLEDGE INSTANT" CHEAT SHEET

DAY 1 INSTANT RECALL COMMAND CHEAT SHEET	
Linux Navigation & Hardware	• pwd • ls -lh / ls -la • cd ~/hpc_day01 && cd .. • lscpu | grep -E "Model name|Socket|Core|Thread" • free -h • numactl -H (inspect NUMA memory nodes)
Filesystem & Permissions	• mkdir -p ~/hpc_day01/{scripts,logs,bin,data} • chmod 750 run.sh (rwxr-x---) • chmod 600 id_ed25519 (rw------- private key) • cat /proc/cpuinfo | grep "flags" | head -n 1
Shell Automation	• #!/usr/bin/env bash (shebang line 1) • set -euo pipefail (strict error abort) • VAR="value" (NO SPACES around =) • echo "Status: $?" (inspect process exit code) • cmd > out.log (overwrite) / cmd >> out.log (append)
Cryptographic Remote SSH	• ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519 • cat ~/.ssh/id_ed25519.pub >> ~/.ssh/authorized_keys • chmod 600 ~/.ssh/authorized_keys • ssh -i ~/.ssh/id_ed25519 user@hpc-node
Intel oneAPI & MPI Discovery	• source /opt/intel/oneapi/setvars.sh 2>/dev/null • icpx --version / icx --version • mpicxx -O3 app.cpp -o app • mpirun --allow-run-as-root --oversubscribe -n 4 ./app

---

📋 DAY 2: COMPLETE 6-HOUR OPERATIONAL PLAYBOOK

Theme: Parallel Computing, OpenMP Multithreading, Hardware False Sharing, MPI & oneDAL
Official Reference: Topics 1–5 on Page 2 of `Intel & NEC - High Performance Computing.pdf`

👷 Morning Session (09:30 AM - 12:30 PM | 3 Hours): Parallel Theory & Memory Physics

- **09:30 - 10:15 (45m) | Module 8: Parallel Computing Principles & Amdahl's Law**
  - *Trainer Delivery:* The Power Wall and why single-core clock scaling stopped at ~4 GHz.
  - *Mathematical Derivation:* $S(N) = \frac{1}{(1-P)+P/N}$. Prove why adding more cores yields diminishing returns when serial bottlenecks exist. Contrast Task Parallelism vs Data Parallelism.

- **10:15 - 11:00 (45m) | Module 9: OpenMP Shared-Memory Architecture**
  - *Trainer Delivery:* The Fork-Join model. Master thread spawns worker threads inside `#pragma omp parallel`.
  - *Data Scoping:* `shared` vs `private` vs `firstprivate`. How race conditions occur during accumulator summation and how the `reduction(+:sum)` clause resolves them safely.
- **11:00 - 11:15 (15m) | ☕ Rejuvenation Break**
- **11:15 - 11:45 (30m) | Module 10: Silicon False Sharing & the MESI Cache Protocol**
  - *Trainer Delivery:* What is a 64-byte cache line? When two CPU cores concurrently write to adjacent indices in an unpadded array, hardware MESI cache invalidation causes memory latency to explode from 1ns to 80ns!
  - *The Hardware Solution:* Enforcing cache alignment via `alignas(64) struct ThreadData { int64_t sum; char pad[56]; };`.
- **11:45 - 12:30 (45m) | Modules 11, 12 & 13: MPI Distributed Systems & Intel oneDAL**
  - *Trainer Delivery:* Distributed-memory systems. Processes communicate across physical nodes using message passing (`MPI_Init`, `MPI_Comm_rank`, `MPI_Reduce`).
  - *Intel AI Analytics Toolkit:* How `sklearnex` intercepts standard Scikit-Learn calls (`patch_sklearn()`) to route mathematical kernels through Intel oneDAL and AVX-512 vector pipelines.

💻 Afternoon Session (01:30 PM - 04:30 PM | 3 Hours): Server & Colab Hands-On Lab

- **01:30 - 02:15 (45m) | Lab Stage 1: OpenMP Thread Scaling Benchmark (50M Vector Sum)**
  - Students compile and run `openmp_vector_sum.cpp` across 1, 2, 4, and 8 threads.
  - Measure real wallclock time and calculate live speedup vs theoretical linear scaling.
- **02:15 - 03:00 (45m) | Lab Stage 2: Live False Sharing Silicon Benchmark (`false_sharing_demo.cpp`)**
  - Run the unpadded array benchmark vs the `alignas(64)` cache-aligned benchmark.
  - Observe the dramatic **3x–6x execution speedup** directly caused by eliminating cache thrashing.
- **03:00 - 03:45 (45m) | Lab Stage 3: MPI Multi-Process Distributed Collective Reduction**
  - Compile and execute `mpi_collective_reduction.cpp` across 4 isolated ranks:

    ```
    mpicxx -O3 mpi_collective_reduction.cpp -o ~/hpc_day02/bin/mpi_reduction
    mpirun --allow-run-as-root --oversubscribe -n 4 ~/hpc_day02/bin/mpi_reduction
    ```

  - Confirm that Rank 0 aggregates partial sums computed by Ranks 1, 2, and 3.
- **03:45 - 04:30 (45m) | Lab Stage 4 & 5: Intel oneDAL 500,000-Sample K-Means & Supervised ML**
  - Execute `benchmark_kmeans_sklearnex.py` on 500k samples with 32 features.
  - Observe unpatched Scikit-Learn (~45 seconds) vs Intel oneDAL (~2.5 seconds): **18x–22x speedup**!
  - Wrap-up with the Day 2 Cheat Sheet and assessment questions.

📌 DAY 2 "ISKO LAGA DAALA TOH 80-90% KNOWLEDGE INSTANT" CHEAT SHEET

DAY 2 INSTANT RECALL COMMAND CHEAT SHEET	
OpenMP Multithreading	• #pragma omp parallel (fork-join team) • #pragma omp parallel for reduction(+:sum) • omp_get_thread_num() (thread ID 0 to N-1) • omp_get_num_threads() (total team size) • export OMP_NUM_THREADS=8 • g++ -O3 -fopenmp app.cpp -o app
Cache Line False Sharing	• Cache line size = exactly 64 bytes • alignas(64) struct ThreadData { int sum; char p[60];} • Eliminates hardware MESI bus invalidation thrashing
MPI Distributed Systems	• MPI_Init(&argc, &argv) / MPI_Finalize() • MPI_Comm_rank(MPI_COMM_WORLD, &rank) • MPI_Comm_size(MPI_COMM_WORLD, &size) • MPI_Reduce(sbuf, rbuf, count, MPI_INT, MPI_SUM, 0...) • mpirun --allow-run-as-root --oversubscribe -n 4 ./app
Intel oneDAL Machine Learning	• from sklearnex import patch_sklearn; patch_sklearn() • from sklearn.cluster import KMeans • 10x-30x faster K-Means, SVM, PCA on Intel CPUs

---

📋 DAY 3: COMPLETE 6-HOUR OPERATIONAL PLAYBOOK

Theme: Deep Learning on HPC, oneDNN Operator Fusion, Slurm Scheduling & Horovod Distributed AI
Official Reference: Topics 1–7 on Page 2 of `Intel & NEC - High Performance Computing.pdf`

👷 Morning Session (09:30 AM - 12:30 PM | 3 Hours): DL Systems & Cluster Orchestration

- **09:30 - 10:15 (45m) | Module 14 & 15: Deep Learning Workflows & Hardware-Aware Optimization**
  - *Trainer Delivery:* The computational anatomy of deep neural networks: forward pass, loss calculation, backpropagation, and weight update.
  - *Memory Wall in Deep Learning:* Why training large models chokes memory bandwidth. Computing vs Memory bound operations.
- **10:15 - 11:00 (45m) | Module 16 & 17: Intel oneDNN Operator Fusion & Supercomputer Accelerators**
  - *Trainer Delivery:* What is oneDNN? Explaining Operator Fusion: fusing Convolution + BatchNormalization + ReLU into a single cache-resident kernel.
  - *Hardware Engines:* Intel AMX (Advanced Matrix Extensions) on 4th/5th Gen Xeon processors computing 1024 BF16 operations/clock cycle. Habana Gaudi 2/3 systolic Matrix Multiplication Engines (MMEs) with on-chip 24x 100GbE RoCE.
- **11:00 - 11:15 (15m) | ☕ Rejuvenation Break**
- **11:15 - 12:00 (45m) | Module 18: Slurm Workload Manager Architecture & Policies**
  - *Trainer Delivery:* Supercomputer architecture: Login Node vs Compute Nodes. Why users must NEVER run heavy workloads on the login node.
  - *Slurm Daemons:* `slurmctld` (controller on head node) and `slurmd` (worker daemon on compute nodes).

- *Directives:* Deconstructing `#SBATCH --nodes=2`, `--ntasks-per-node=4`, `--cpus-per-task=8`, `--time=02:00:00`.

- **12:00 - 12:30 (30m) | Modules 19 & 20: Distributed Training Mechanics & Horovod**
  - *Trainer Delivery:* Parameter Server architecture vs Decentralized Ring-AllReduce. Why Parameter Servers bottleneck at scale, and how Baidu/Horovod Ring-AllReduce eliminates the central bottleneck.

💻 Afternoon Session (01:30 PM - 04:30 PM | 3 Hours): Server & Colab Hands-On Lab

- **01:30 - 02:15 (45m) | Lab Stage 1: Intel oneDNN Operator Fusion Benchmark**
  - Execute the TensorFlow oneDNN layer fusion benchmark comparing unfused sequential layers vs fused kernels on synthetic image batches.
  - Observe memory throughput improvements and CPU thread scaling.

- **02:15 - 03:00 (45m) | Lab Stage 2: Slurm Discrete-Event Job Scheduling Simulator (`slurm_simulator.py`)**
  - Run the in-memory Slurm queue simulator modeling cluster partitions (`cpu-standard`, `gpu-gaudi`), Fair-Share priority queues, job states (`PD`, `R`, `CD`), and node allocation.

- **03:00 - 03:45 (45m) | Lab Stage 3: Enterprise Slurm Batch Scripting (`train_resnet_hpc.slurm`)**
  - Write and audit an enterprise `#SBATCH` batch script for distributed ResNet training:

```
cat << 'EOF' > ~/hpc_day03/scripts/train_resnet_hpc.slurm
#!/usr/bin/env bash
#SBATCH --job-name=resnet50_hpc
#SBATCH --output=logs/%x_%j.out
#SBATCH --error=logs/%x_%j.err
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=4
#SBATCH --cpus-per-task=8
#SBATCH --partition=compute-ai
#SBATCH --time=01:00:00

source /opt/intel/oneapi/setvars.sh 2>/dev/null || true
export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
srun python3 train_model.py
EOF
```

- **03:45 - 04:30 (45m) | Lab Stage 4 & 5: Horovod Multi-Process Distributed Training & MCQs**
  - Launch 4 parallel distributed training workers via OpenMPI simulating cluster training with Horovod learning rate scaling ($lr \times \text{size}$).
  - Measure training images/sec throughput scaling and review the Day 3 Cheat Sheet.

📌 DAY 3 "ISKO LAGA DAALA TOH 80-90% KNOWLEDGE INSTANT" CHEAT SHEET

DAY 3 INSTANT RECALL COMMAND CHEAT SHEET	
Intel oneDNN Deep Learning	• export TF_ENABLE_ONEDNN_OPTS=1 • Fuses Conv2D + BatchNorm + ReLU into CPU L1 cache • Activates Intel AMX tile registers on Xeon CPUs
Slurm Workload Manager	• sbatch script.slurm (submit batch job to queue) • squeue -u $USER (view running and pending jobs) • sinfo (view cluster partitions and node states) • scancel <job_id> (cancel running or queued job) • srun -n 4 hostname (spawn parallel interactive task)
Core #SBATCH Directives	• #SBATCH --nodes=2 (number of physical compute nodes) • #SBATCH --ntasks-per-node=4 (MPI tasks per node) • #SBATCH --cpus-per-task=8 (CPU threads per task) • #SBATCH --output=logs/%x_%j.out (%x=name, %j=job_id)
Horovod Distributed AI	• import horovod.tensorflow as hvd; hvd.init() • opt = hvd.DistributedOptimizer(opt) • opt.lr = base_lr * hvd.size() (learning rate scaling) • Decentralized Ring-AllReduce gradient synchronization

---

📋 DAY 4: COMPLETE 6-HOUR OPERATIONAL PLAYBOOK

Theme: Distributed AI Scaling, Ring-AllReduce Math, Intel VTune TMAM & Advisor Roofline Modeling
Official Reference: Topics 1–2 on Page 2 of `Intel & NEC - High Performance Computing.pdf`

👷 Morning Session (09:30 AM - 12:30 PM | 3 Hours): Distributed Scaling & Profiling Theory

- **09:30 - 10:15 (45m) | Module 21 & 22: Distributed AI with Slurm & Ring-AllReduce Mechanics**
  - *Trainer Delivery:* The mathematical proof of Ring-AllReduce communication volume.
  - *Equation:* Total data transferred per node is strictly $2 \times \frac{N-1}{N} \times S$, completely independent of the number of nodes N!
- **10:15 - 11:00 (45m) | Module 23 & 24: Intel VTune Profiler & TMAM Microarchitecture Analysis**
  - *Trainer Delivery:* Top-Down Microarchitecture Analysis Method (TMAM).
  - *The 4 Core Buckets:* Retiring (useful work), Bad Speculation (branch mispredictions), Frontend Bound (instruction fetch/decode stalls), Backend Bound (Memory Bound vs Core Bound stalls).
- **11:00 - 11:15 (15m) | ☕ Rejuvenation Break**
- **11:15 - 12:30 (75m) | Modules 25 & 26: Intel Advisor Roofline Modeling & Cluster Scaling Laws**
  - *Trainer Delivery:* Arithmetic Intensity ($OI = \frac{\text{FLOPs}}{\text{Bytes}}$).
  - *The Roofline Plot:* Logarithmic curve showing when code is bottlenecked by DRAM memory bandwidth vs when code reaches the peak theoretical FLOPS compute ceiling.
  - *Scaling Laws:* Amdahl's Law (strong scaling) vs Gustafson's Law (weak scaling) at 64, 128, and 256 cluster cores.

💻 Afternoon Session (01:30 PM - 04:30 PM | 3 Hours): Server & Colab Hands-On Lab

- **01:30 - 02:30 (60m) | Lab Stage 1 & 2: Multi-Node Slurm Distributed AI & Ring-AllReduce Emulation**
  - Generate multi-node Slurm job configurations requesting 4 nodes (16 ranks total).
  - Run the verified Python Ring-AllReduce simulation validating ring data transfers with zero bottlenecks.
- **02:30 - 03:30 (60m) | Lab Stage 3 & 4: VTune TMAM Diagnostic Analyzer & Hotspot Profiling**
  - Compile unoptimized cache-thrashing matrix kernels and run the TMAM level 1/2 microarchitecture classifier.
  - Identify memory latency stalls and refactor into cache-blocked tiled loops.
- **03:30 - 04:30 (60m) | Lab Stage 5 & 6: Interactive Advisor Roofline Generator & MCQs**
  - Generate interactive Matplotlib Roofline curves comparing unoptimized kernels ($OI = 0.125$) vs AVX2/AVX-512 tiled kernels ($OI = 16.0$).
  - Review the Day 4 Cheat Sheet and assess student mastery.

📌 DAY 4 "ISKO LAGA DAALA TOH 80-90% KNOWLEDGE INSTANT" CHEAT SHEET

DAY 4 INSTANT RECALL COMMAND CHEAT SHEET	
Ring-AllReduce Formula	• Total transfer = 2 * ((N - 1) / N) * S bytes • Independent of node count N -> scales to 1,000s nodes
Intel VTune TMAM Profiling	• Retiring: CPU executing useful retired uOps • Bad Speculation: Branch mispredictions throwing uOps • Frontend Bound: L1i cache misses / instruction fetch • Backend Bound: Memory Bound (DRAM/LLC) vs Core Bound
Intel Advisor Roofline Model	• Arithmetic Intensity = Total FLOPs / Total Bytes read • Below knee (< 1.0 FLOP/byte) = strictly Memory Bound • Above knee (> 10.0 FLOP/byte) = Peak Compute Bound
Cluster Scaling Laws	• Strong Scaling: Fixed problem size, more cores • Weak Scaling: Scaled problem size, constant per-core

---

📋 DAY 5: COMPLETE 6-HOUR OPERATIONAL PLAYBOOK

Theme: Generative AI, Foundation LLMs, Habana Gaudi 2/3, OpenVINO IR, NNCF INT4 & FastAPI
Official Reference: Topics 1–4 on Page 2 of `Intel & NEC - High Performance Computing.pdf`

👷 Morning Session (09:30 AM - 12:30 PM | 3 Hours): GenAI Systems & Silicon Architecture

- **09:30 - 10:15 (45m) | Module 27: Generative AI & Foundation LLMs on HPC**
  - *Trainer Delivery:* The Transformer architecture: Self-Attention, Key-Value (KV) cache, Prefill phase vs Autoregressive Decode phase.
  - *Why LLM Decode is Memory-Bound:* Generating 1 token requires streaming all model weights across the memory bus ($O(1)$ arithmetic intensity).
- **10:15 - 11:00 (45m) | Module 28: Intel Hardware for LLMs: Xeon AMX & Habana Gaudi 2/3**

- *Trainer Delivery:* Intel AMX tile matrix acceleration on Xeon CPUs.
- *Habana Gaudi Architecture:* Dual Matrix Multiplication Engines (MMEs), Tensor Processor Cores (TPCs), and 24x 100GbE integrated RoCE v2 ports eliminating external InfiniBand cards.

- **11:00 - 11:15 (15m) | ☕ Rejuvenation Break**
- **11:15 - 12:30 (75m) | Modules 29 & 30: Intel OpenVINO IR Export & NNCF INT4 Quantization**
  - *Trainer Delivery:* OpenVINO Intermediate Representation (`.xml` computational graph + `.bin` weights).
  - *Quantization Physics:* Compressing FP16 (2 bytes/weight) to INT4 (0.5 bytes/weight). Reducing memory bandwidth pressure by **75%**, resulting in a **3x–4x decode speedup**!

💻 Afternoon Session (01:30 PM - 04:30 PM | 3 Hours): Server & Colab Hands-On Lab

- **01:30 - 02:15 (45m) | Lab Stage 1 & 2: Hardware AMX Audit & OpenVINO IR Export**
  - Verify CPU flags (`amx_tile` , `amx_int8` , `amx_bf16`) in `/proc/cpuinfo` .
  - Export causal LLMs (`TinyLlama` / `Qwen2.5`) to native OpenVINO IR representation.
- **02:15 - 03:00 (45m) | Lab Stage 3 & 4: NNCF INT4 Compression & Real-Time Streaming Benchmark**
  - Execute `compress_llm_nncf.py` compressing model weights to INT4.
  - Measure Time-to-First-Token (TTFT), Inter-Token Latency (ITL), and real-time Tokens-Per-Second (TPS) streaming generation.
- **03:00 - 03:45 (45m) | Lab Stage 5 & 6: Multi-Node Slurm LoRA Fine-Tuning & FastAPI Microservice**
  - Audit `sbatch_finetune_llm.sh` with DeepSpeed ZeRO-2/3 parameter sharding on Habana Gaudi / Xeon clusters.
  - Deploy the OpenVINO-accelerated LLM as an asynchronous REST microservice (`llm_serving_api.py`) with `/health` and `/api/generate` endpoints tested via `TestClient` .
- **03:45 - 04:30 (45m) | Lab Stage 7: Masterclass Capstone Assessment & Final Certification Wrap-Up**
  - Review the Day 5 Cheat Sheet.
  - Conduct the final architectural mastery evaluation.

📌 DAY 5 "ISKO LAGA DAALA TOH 80-90% KNOWLEDGE INSTANT" CHEAT SHEET

DAY 5 INSTANT RECALL COMMAND CHEAT SHEET	
LLM Autoregressive Bottleneck	• Prefill phase: Compute-bound (matrix multiplication) • Decode phase: Strictly Memory-bound (reads all weights) • Memory bandwidth is the primary speed limiter
Habana Gaudi Differentiator	• 24x 100GbE integrated RoCE v2 ports on silicon die • Direct all-reduce across cluster with NO PCIe NIC bottleneck
OpenVINO IR & NNCF INT4	• OpenVINO IR = model.xml (graph) + model.bin (weights) • nncf.compress_weights(mode=CompressWeightsMode.INT4) • Cuts weight footprint from 2.2 GB down to 640 MB • 3x-4x faster token decode throughput on Intel CPUs
Production FastAPI Serving	• @app.post("/api/generate") -> Streaming token output • @app.get("/health") -> Hardware & INT4 telemetry • uvicorn.run(app, host="127.0.0.1", port=8000)

🏆 SUMMARY OF MASTERCLASS RIGOR

1. **6 Full Hours per Day:** Every day is fully realized with 3 Hours of Morning Theory & System Architecture and 3 Hours of Hands-on Lab on University Linux Servers and Google Colab.
2. **100% Syllabus Adherence:** Strictly bounded by `Intel & NEC - High Performance Computing.pdf`.
3. **Module-by-Module Arrow Workflows:** Every single concept has its accompanying visual tree and hardware mapping.
4. **Instant Recall Cheat Sheets:** High-impact command reference tables giving students 80-90% instant operational recall.
5. **Dual-Vector Execution:** Flawlessly runs on university cluster terminals via SSH AND on Google Colab cloud sandboxes with zero friction.